Faculty: CSIT
I/II
Subject : OOP

Instructor

Er.Nirajan Dhaurali

HIMALAYA COLLEGE OF ENGINEERING

Don’t stop when you’re tired. Stop when you’re done.

– Wesley Snipes

Unit 3. Classes & Objects

- A Simple Class and Object
- Accessing members of class
- Initialization of class objects: (Constructor, Destructor)
  - Default Constructor
  - Parameterized Constructor
  - Copy Constructor
  - The Default Copy Constructor
- Objects as Function Arguments
- Returning Objects from Functions
- Structures and Classes
- Memory allocation for Objects
- Static members
- Member functions defined outside the class.

Class

- Object-oriented programming (OOP) encapsulates data (attributes) and functions (behavior) into a single unit called classes
- A class is a user-defined data type that contains data members and member functions.
- Class is a blueprint of real world objects. Which describes object's property and behaviors

Syntax :

```
class class_name
{
   access specifier :
   //data members
   access specifier :
   //member functions
};
```

```
class Student {
    private:          //access specifier
    int rollNumber;          // Data member
    int marks;

    public:                                 //access specifier
    void setProperty(int r, int m)     // Member function
    {
        rollNumber=i;
        marks=m;
    }
    void display()
    {
        cout << "ROll Number: " << rollNumber<<endl;
        cout ,, "Marks: " << marks<<endl;
    }
};
```

- The body of class is terminated by a semicolon.
- The class body contains the declaration of variables and functions.
- The functions and variables are collectively called members.
- The variables declared inside a class are known as data members.
- The functions declared inside a class are known as member functions.

Naming conventions :

- variables,methods – camelCase
- ClassName – PascalCase
- filename: smallercase

Access Specifiers / Access Modifiers

- It describes how the members of the class can be accessed

Three available access specifiers are:

i. **Private:** The members declared as private are accessible only within the class. Private data members are used only by member functions. When anything is not specified it is private by default.

ii. **Public:** The members declared as public are accessible from outside the class.

iii. **Protected:** The members declared as protected are accessible from outside the class but only in derived class from it(will be discussed in Inheritance). If there is no inheritance used, protected is similar to private.

```
class Customer {
    int cust_id; // private by default
    char cust_name[10]; // private by default
public:
    void get_inp(); // public
    void display(); // public
private:
    char address[10]; // private
protected:
    char tel[10]; // protected
public:
    float discount;
};
```

Object:

- An object is a real-world entity or an instance of a class that holds the data (variables) declared in the class, and the member functions operate on these class objects.
- Once class has been defined, objects can be created from the class
- We can create any number of objects from the same class

Syntax

- ClassName ObjectName;

Eg:

- Student S1,S2,S3; *//Here , S1,S2,S3 are objects of class Student*
- Customer C1,C2; *//Here , C1,C2 are objects of class Customer*
- Rectangle R1,R2; *//Here , R1,R2 are objects of class Rectangle*

Alternative way to create object :

```
Class Student
{
   // class definition
} S1,S2,S3;
```

Object and member access

- Since class is just a blueprint or template for the object, While defining a class, no storage is assigned except for the definition of member function.
- Objects are instances of class which holds the data variables declared in the class and member functions work on these class objects.
- Each object has different data variables and shares common functions.

function_1()		
function_2()		
object1	object2	object3
var_1	var_1	var_1
var_2	var_2	var_2

Accessing Member of class through Object:

- When an object of the class is created then the members are accessed using the **(.)** dot operator.

Syntax :

- ObjectName.data_member;
- ObjectName.member_function();

Syntax :

- S1.roll_no= 1;
- S1.marks=500;
- R1.area();

Note :

The dot (.) operator can only access public members of a class from outside.

For private and protected members, you must use public interface methods or inheritance in case of protected

A complete Example:

```cpp
#include <iostream>
using namespace std;

class Rectangle {
private:
    int length, breadth;

public:
    void setData(int l, int b) {
        length = l;
        breadth = b;
    }
    void showData() {
        cout << "Length = " << length << endl;
        cout << "Breadth = " << breadth << endl;
    }
    int findArea() {
        return length * breadth;
    }
    int findPerimeter() {
        return 2 * (length + breadth);
    }
};
int main() {
    Rectangle r;
    r.setData(4, 2);
    r.showData();
    cout << "Area = " << r.findArea() << endl;
    cout << "Perimeter = " << r.findPerimeter();
    return 0;
}
```

```
Length = 4
Breadth = 2
Area = 8
Perimeter = 12
```

Defining member function outside class:

- In C++, member functions of a class can be defined outside the class using the (**::**) scope resolution operator
- This keeps the class definition clean and improves code organization

Syntax :

```
return_type ClassName :: function_name(argument)
{
    //Function body
}
```

Example :

```
#include<iostream>
using namespace std;
class rectangle{
private:
int length, breadth;
public:
void setdata(int l, int b);
void showdata();
int findarea();
int findperimeter();
};

void rectangle :: setdata(int l, int b){
length = l; breadth = b;
}
void rectangle :: showdata(){
cout<<"Length = "<<length<<endl<<"Breadth = "<<breadth<<endl;
}
int rectangle :: findarea(){
return length * breadth;
}
int rectangle :: findperimeter(){
return (2 * (length + breadth));
}
int main(){
rectangle r;
r.setdata(5, 3);
r.showdata();
cout<<"Area= "<<r.findarea()<<endl;
cout<<"Perimeter= "<<r.findperimeter();
return 0;
}
```

```
Length = 5
Breadth = 3
Area= 15
Perimeter= 16
```

Initialization of class object:

Constructor

- In previous slides, values for the data members have been provided by using member functions. e.g. void setdata()
- Providing value using those setters functions does not conform with the philosophy of C++ language.
- class type variables i.e. objects can also be initialized in similar way using Constructor
- **A constructor is a special member function whose task is to initialize the objects of its class.**
- **The name of constructor is same as the name of class**
- **It has no return type, so can't use return keyword**
- **It is used for automatic initialization**

Syntax:

```
class ClassName{
private:
//data members
public:
ClassName()
{
    //statements
}
//other member functions
};
```

Characteristics of the Constructor

- It is a member function with same name as that of Class and does not have return type.
- They should be declared in the public section.
- They are called automatically when the objects are created.
- They do not have return types, not even void so, they cannot return values.
- They can not be inherited (other member functions can be), though a derived class can call the base class constructor. (will be discussed in Inheritance section)
- Like other C++ functions, they can have default arguments.
- Constructors cannot be virtual. Virtual Functions will be discussed in Inheritance/Polymorphism
- Constructors can also be overloaded but not destructor
- All the constructor overloading rules are same as for other C++ functions.

Types of Constructor

1.Default coustructor

- A constructor that accepts no parameters is called the default constructor.
- If no such constructor is defined, then the compiler supplies a default constructor.
- If programmer defines the default constructor, compiler will not create.

Eg:

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int length, breadth;
public:
    // Default Constructor
    Rectangle() {
        length = 5;      // Default length
        breadth = 3;     // Default breadth
    }

    // Member Function to calculate area
    int findArea() {
        return length * breadth;
    }
};
int main() {
    Rectangle r;  // Constructor is automatically called
    cout << "Area = " << r.findArea() << endl;
    return 0;
}
```

2.Parameterize constructor

- A constructor that accepts parameters is called the parameterized constructor.
- When an object for a class is created, initial values as arguments to the constructor can be passed similar to function call.
- Parameterized Constructor can be **called** in two ways:
- Implicitly

    Eg: Rectangle r1(5,3)

- Explicitly

    Eg: Rectangle r1= Rectangle(5,3)

```cpp
#include <iostream>
using namespace std;
class Rectangle {
private:
    int length, breadth;

public:
    // Parameterized Constructor with default values
    Rectangle(int l = 1, int b = 1) {
        length = l;
        breadth = b;
    }
    int findArea() {
        return length * breadth;
    }
};

int main() {
    Rectangle r1;        // r1: called with default values

    Rectangle r2(4, 3); // r2: called implicitly

    Rectangle r3 = Rectangle(6, 5); // r3: called explicitly

    cout << "Area of Object r1:"<< r1.findArea()<<endl;
    cout << "Area of Object r2:"<< r2.findArea()<<endl;
    cout << "Area of Object r3:"<< r3.findArea()<<endl;

    return 0;
}
```

3.Copy constructor

- A copy constructor is used to declare and initialize object from another abject
- the process of initializing members of an object through copy constructor is known as copy initialization

Defining a copy constructor

```
Rectangle ( Rectangle &r )
{
length = r.length;
breadth= r.breadth;
}
```

Calling a copy constructor

```
Rectangle r1;
```

Rectangle r2(r1) or Rectangle r2=r1

```
/*** COPY CONSTRUCTOR ***/
#include <iostream>
using namespace std;
class Box {
    int length;
public:
    Box(int l) {          //parameterized constructor
    length = l;
    }
    Box(Box &b){          //copy constructor
    length=b.length;
    }
    void display() {
        cout << "Length = " << length << endl;
    }
};
int main() {
    Box b1(10);         // parameterized constructor called
    cout << "Box b1: ";
    b1.display();

    Box b2 = b1;        // copy constructor is called
    cout << "Box b2 (copied from b1): ";
    b2.display();
    return 0;
}
```

4. Default Copy constructor

- A default copy constructor is an automatically provided constructor by the C++ compiler that creates a copy of an object.

```cpp
#include <iostream>
using namespace std;

class Box {
    int length;

public:
    void setLength(int L) {
        length = L;
    }

    void display() {
        cout << "Length = " << length << endl;
    }
};

int main() {
    Box b1;         // Calls parameterized constructor
    b1.setLength(10);
    cout << "Box b1: ";
    b1.display();

    Box b2 = b1;       // Default copy constructor is called
    cout << "Box b2 (copied from b1): ";
    b2.display();
    return 0;
}
```

Destructor

- It is used to destroy the objects that have been created by a constructor.
- Like a constructor, the destructor is a member function whose name is the same as the class name but is preceded by a tilde (~)
- **~A(){}** is a destructor for class **A**
- A destructor never takes any argument nor does it return any value.
- Most common use of destructor is to destroy the memory that was allocated for object by constructor
- Destructor can’t be overload (i.e only one destructor per class)
- Destructor Called in Reverse Order of Constructor

```cpp
#include <iostream>
using namespace std;
class Demo {
public:
    // Constructor
    Demo() {
        cout << "Constructor is called." << endl;
    }
    // Destructor
    ~Demo() {
        cout << "Destructor is called." << endl;
    }
};

int main() {
    {
        Demo d; // Object is created — constructor will be called
    } // Object goes out of scope — destructor will be called automatically
    cout << "Back in main function." << endl;

    return 0;
}
```

Destructor Called in Reverse Order of Constructor:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    A() {
        cout << "Constructor of Class A called." << endl;
    }
    ~A() {
        cout << "Destructor of Class A called." << endl;
    }
};
class B {
public:
    B() {
        cout << "Constructor of Class B called." << endl;
    }
    ~B() {
        cout << "Destructor of Class B called." << endl;
    }
};

int main() {
    A a;
    B b;
    cout << "Inside main function." << endl;
    return 0;
}
```

```
Constructor of Class A called.
Constructor of Class B called.
Inside main function.
Destructor of Class B called.
Destructor of Class A called.
```

Constructor overloading

- Constructor overloading means defining multiple constructors in the same class
- This allows an object to be initialized in different ways depending on the arguments passed

Constructor can be overloaded by :

1. Different number of arguments
2. Different types of arguments

```cpp
#include <iostream>
using namespace std;
class Box {
    int length, breadth;

public:
    Box() {                  // Default constructor
        length = 0;
        breadth = 0;
    }
    Box(int l) {             // Parameterized constructor (1 parameter)
        length = l;
        breadth = 0;
    }
    Box(int l, int b) {      // Parameterized constructor (2 parameters)
        length = l;
        breadth = b;
    }
    void display() {
        cout << "Length = " << length << ", Breadth = " << breadth << endl;
    }
};
int main() {
    Box b1;             // Calls default constructor
    Box b2(5);          // Calls constructor with 1 parameter
    Box b3(4, 7);       // Calls constructor with 2 parameters

    b1.display();
    b2.display();
    b3.display();
    return 0;
}
```

Class Work:

1. Write a C++ program to define a class Student with data members name and age, and a member function display(). Use an object to display the student details
2. Create a class Employee to: Store name, id, and salary. Use a seter function to initialize these values. Write a display function to display employee details.
3. Create a class Circle that uses: A default constructor to initialize radius to 0. A parameterized constructor to initialize radius with given values. A member function to calculate and display area.
4. Create a class Item that: Demonstrates use of a copy constructor. Initialize object i1 using a parameterized constructor. Initialize object i2 using a copy of i1. Display data for both objects.
5. Create a class Student with the following: A constructor to initialize name and roll number. A display() function to print student details. Use constructor overloading to create objects with and without roll numbers.

Me: I will study computer science and become hacker in future.

Teacher: What is computer..?

Me:

Object as function argument

- Like any other data type, an object can also be used as function argument.
- Object can be passed as arguments in three ways:

1. Pass-by-Value

- In this method a copy of the object is passed to the function.
- Any changes made to the object inside the function do not affect the object used in the function call.

2. Pass-by-Reference

- In this method an address of the object is passed to the function.
- Any changes made to the object inside the function will reflect in the actual object.

3. Pass-by-Pointer

- pass-by-pointer method can also be used to work directly on the actual object used in the function call.

```
/****** PASS BY VALUE *****/

#include <iostream>
using namespace std;

class Box {
    int length;

public:
    Box(int l) {
        length = l;
    }

    void compare(Box b) { // Pass by value
        if (length == b.length)
            cout << "both Boxes are equal in length." << endl;
        else
            cout << "Boxes are not equal in length." << endl;
    }
};

int main() {
    Box b1(10);
    Box b2(10);
    b1.compare(b2); // Copy of b2 passed
    return 0;
}
```

```cpp
/****** PASS BY REFERENCE *****/

#include <iostream>
using namespace std;

class Box {
    int length;

public:
    Box(int l) {
        length = l;
    }

    void compare(Box &b) { // Pass by reference
        if (length == b.length)
            cout << "Both Boxes are equal in length." << endl;
        else
            cout << "Boxes are not equal in length." << endl;
    }
};

int main() {
    Box b1(12);
    Box b2(15);
    b1.compare(b2); // Reference of b2 passed
    return 0;
}
```

```cpp
/****** PASS BY POINTER *****/

#include <iostream>
using namespace std;

class Box {
    int length;

public:
    Box(int l) {
        length = l;
    }

    void compare(Box *b) { // Pass by address
        if (length == b->length)
            cout << "Boxes are equal in length." << endl;
        else
            cout << "Boxes are not equal in length." << endl;
    }
};

int main() {
    Box b1(20);
    Box b2(20);
    b1.compare(&b2); // Address of b2 passed
    return 0;
}
```

Returning Object from function

- A function cannot only receive objects as arguments but also can return them.
- Syntax: ClassName Object_Name;
- A function can return objects by the following three ways:

1. **Return-by-Value**
2. **Return-by-Reference**
3. **Return-by-Pointer**

Q1. WAP to add two complex numbers using the concept of passing an object to a function and returning an object from the function.
Q2. Explain the concept of passing objects to a function and returning object from a function with an example

```
/****** ADD TWO COMPLEX NUMBER *****/

#include <iostream>
using namespace std;

class Complex {
    float real;
    float imag;

public:
    Complex(float r = 0, float i = 0) {      // Constructor
        real = r;
        imag = i;
    }

    Complex add(Complex c) {             // Function to add two complex numbers
        Complex temp;
        temp.real = real + c.real;
        temp.imag = imag + c.imag;
        return temp;
    }
    void display() {
        cout << "Result = " << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(2, 3);
    Complex c2(1, 4);

    Complex c3;
    c3 = c1.add(c2); // Passing object c2 to c1's add function

    c3.display(); // Output: Result = 3 + 7i
    return 0;
}
```

NOTE:

We can write :

```
Complex(float r = 0, float i = 0) {
    real = r;
    imag = i;
}
```



```
Complex(float r = 0, float i = 0) : real(r), imag(i) {}
```

```
Complex add(Complex c) {
    Complex temp;
    temp.real = real + c.real;
    temp.imag = imag + c.imag;
    return temp; // Returning object
}
```



```
Complex add(Complex c) {
  return Complex(real + c.real, imag + c.imag);
}
```

Structures and Classes in C++

- In C++, both structures (struct) and classes (class) are used to define user-defined data types that can hold both data members and member functions. The main difference between them lies in the default access specifier.

1. Structure (struct)

- In struct, members are **public** by default.

In C++, struct is enhanced and can have:

- Data members and Member functions
- Access specifiers (public, private, protected)

2. Class

- In class, members are private by default.

Class support all OOP concepts like:

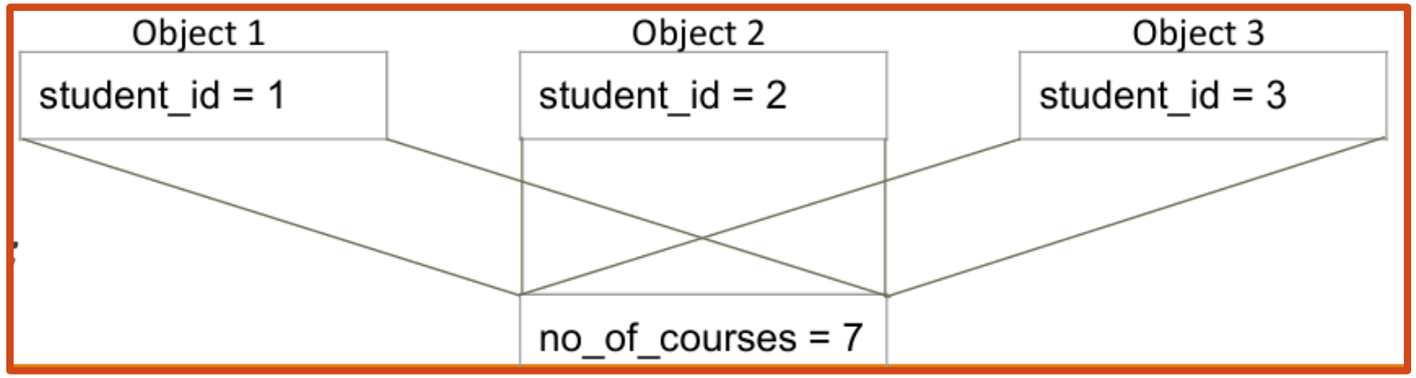
- Encapsulation , Inheritance, Polymorphism, Abstraction

Static Data Members

- If a data member in a class is defined as static, then only one copy of that member is created for the entire class and is shared by all the objects of that class , no matter how many objects are created.
- Hence, these data members are normally used to maintain values common to the entire class and are also called **class variables.**
- Static data member is declared using the **static** keyword inside the class.
- It **must be** defined outside the class as well

Syntax:

```
class A{
static int i;
};
int A:: i=1;
```



```cpp
#include <iostream>
using namespace std;

class Box {
    static int count; // Static data member

public:
    Box() {
        count++; // Increments when a new object is created
    }
    void showCount() {
        cout << "Object count = " << count << endl;
    }
};

// Definition of static member outside the class
int Box::count = 0;

int main() {
    Box b1, b2;
    b1.showCount();       // Output: Object count = 2
    return 0;
}
```

Static Member Function

- Like static member variables, we can also have static member functions.
- Static member function is directly associated with class rather than individual object
- **Eg: static void display()**
- A static member function can have access to only other static members (functions or variables) declared in the same class
- It can be called using the class name (instead of objects) as follows:

  class-name :: function-name;

```cpp
#include <iostream>
using namespace std;

class Box {
    static int count; // Static data member

public:
    Box() {
        count++; // Increments when a new object is created
    }

    // Static member function to access static data
    static void displayCount() {
    cout << "Object count = " << count << endl;
    }
};

// Definition of static member outside the class
int Box::count = 0;

int main() {
    Box b1, b2;
    Box b3;
    Box::displayCount(); // Output:Object count = 3
    return 0;
}
```

Array Of Object:

- An array of objects is just like a normal array, but instead of holding basic data types (like int or float), it holds objects of a class.
- This is useful when you want to create and manage multiple objects of the same class using a single array.
- **Syntax: ClassName objectArrayName[size];**

```cpp
/***** ARRAY OF OBJECTS *******/
#include <iostream>
using namespace std;
class Student {
    int roll;
    string name;
public:
    void setData(int r, string n) {
        roll = r;
        name = n;
    }
    void display() {
        cout << "Roll: " << roll << ", Name: " << name << endl;
    }
};
int main() {
    Student s[3]; // Array of 3 Student objects

    // Setting data for each object
    s[0].setData(1, "Ravi");
    s[1].setData(2, "Sita");
    s[2].setData(3, "Hari");

    // Displaying data
    for (int i = 0; i < 3; i++) {
        s[i].display();
    }
    return 0;
}
```

Constant Object

- A constant object in C++ is an object that cannot be modified after it is created.
- **Syntax: const ClassName obj;**
- The const keyword ensures that the object is read-only.
- Once a const object is declared, you can only call const member functions on it.
- To work with constant objects, member functions must be declared with the const keyword.
- Non-const member functions cannot be called with a const object.

```cpp
#include <iostream>
using namespace std;
class Student {
    int roll;
    string name;
public:
    Student(int r, string n) {
        roll = r;
        name = n;
    }
    // const member function
    void show() const {
        cout << "Roll: " << roll << ", Name: " << name << endl;
    }

    // This function would give error if called on const object
    // void updateRoll(int r) {
    //     roll = r;
    // }
};
int main() {
    const Student s1(101, "Sita");
    s1.show();  // allowed: show() is a const function

    // s1.updateRoll(102); Not allowed: can't modify const object
    return 0;
}
```

this Pointer :

- **this** pointer is a predefined pointer which is used to point the calling object.
- It is used when you want to differentiate between local variables and member variables with the same name.

```
/*******THIS POINTER ******/

    #include <iostream>
    using namespace std;
    class Student {
        int id;
        string name;
    public:
        void setData(int id, string name) {
            // Using 'this' pointer to refer to the current object's members
            this->id = id;
            this->name = name;
        }
        void display() {
            cout << "ID: " << id << ", Name: " << name << endl;
        }
    };
    int main() {
        Student s1;
        s1.setData(101, "Ramesh");
        s1.display();
        return 0;
    }
```

ASSIGNMENT 3

short questions

1. Write a C++ program to display the number of objects created using static member.
2. Define class and object with suitable example. How members of class can be accessed?
3. Write a program to show the destructor call such that it prints the message “memory is released”.
4. What is **this** pointer? How can we use it for name conflict resolution? Illustrate with example.
5. What is the use of **get** and **getline** functions? Explain with suitable example. How getline is differ from cin (explore yourself)

long questions

1. What is constructor and destructor? Why constructor is needed in a class? Illustrate the types of constructor with an example.
2. Write a program for illustrating default constructor, parameterized constructor and copy constructor.
3. What is the benefit of passing object as arguments? Write a program to create a class named actor with data members name and rating. Initialize the data members and display those names whose rating is greater than 5 using the concept of constant object.
4. Write a program according to the specification given below:
   - Create a class Account with data members acc no, balance, and min_balance(static)
   - Include methods for reading and displaying values of objects
   - Define static member function to display min_balance
   - Create array of objects to store data of 5 accounts and read and display values of each object

THANK YOU